

A Comprehensive Study of Algebraic Fault Analysis on PRINCE

ZHANG Fan¹, ZHAO Xinjie², GUO Shize², SHEN Jizhong¹, HUANG Jing³, HU Zijie¹

¹ College of Information Science and Electrical Engineering, Zhejiang University, Hangzhou, China

² The Institute of North Electronic Equipment, Beijing, China

³ System Control Department of No.61541 Unit, Beijing, China

Abstract: PRINCE is a 64-bit lightweight block cipher with a 128-bit key published at ASIACRYPT 2012. Assuming one nibble fault is injected, previous different fault analysis (DFA) on PRINCE adopted the technique from DFA on AES and current results are different. This paper aims to make a comprehensive study of algebraic fault analysis (AFA) on PRINCE. How to build the equations for PRINCE and faults are explained. Extensive experiments are conducted. Under nibble-based fault model, AFA with three or four fault injections can succeed within 300 seconds with a very high probability. Under other fault models such as byte-based, half word-based, word-based fault models, the faults become overlapped in the last round and previous DFAs are difficult to work. Our results show that AFA can still succeed to recover the full master key. To evaluate security of PRINCE against fault attacks, we utilize AFA to calculate the reduced entropy of the secret key for given amount of fault injections. The results can interpret and compare the efficiency of previous work. Under nibble-based fault model, the master key of PRINCE can be reduced to $2^{9.69}$ and $2^{36.10}$ with 3 and 2 fault injections on average, respectively.

Keywords: algebraic fault analysis; PRINCE; CryptoMiniSAT

I. INTRODUCTION

1.1 Background

Fault attacks retrieve the secret information by injecting computational faults into some cryptosystems. Faults can be generated by changing the power supply voltage, changing the frequency of the external clock, varying the environmental temperature or exposing the circuits of the device to intense lights or lasers during the computation [1].

The idea was first reported on RSA-CRT by Boneh *et al* in EUROCRYPT 1997 [3]. The attack was first successfully exploited to break an RSA-CRT with both a correct and a faulty signature of the same message. Shortly after, Biham and Shamir proposed the *differential fault analysis* (DFA) attack, which combines fault attack with differential cryptanalysis [2]. Since then, DFA has been used to break public-key ciphers such as ECC, block ciphers such as AES, ARIA, Camellia and CLEFIA, and stream ciphers such as RC4 and Trivium.

In eSmart 2010, Courtois *et al* [5] proposed a new fault analysis technique known as *algebraic fault attack* (AFA), which combines algebraic cryptanalysis with fault attack. They showed that if 24 key bits are known and two bits in the 13-th round are altered, DES can

be broken with a single fault injection in 0.01 hour. The full attack is 10 times faster than the brute force attack. In COSADE 2013, Zhang *et al* proposed an efficient AFA technique, which showed that the secret key of Piccolo and DES can be recovered with only a single fault injection and less time complexity [19]. Meanwhile, AFA was also applied to improve DFA on LED and Trivium [11].

1.2 Related work on PRINCE

Recent years, many lightweight block ciphers have been proposed in the literature to provide cryptographic building blocks for resource constrained devices. Among the best studied ciphers are PRESENT, Piccolo, LED, MIBS and PRINTCipher. PRINCE is a novel lightweight block cipher proposed in 2012 [7], which is optimized with respect to latency when implemented in hardware. This is the first lightweight block cipher that takes latency as main priority.

Li *et al* [10] mounted improved meet-in-the-middle attacks on AES-192 and PRINCE. They first presented attacks on 8-round PRINCE_{core} and PRINCE with about 2^{53} and 2^{60} encryptions, respectively. Furthermore, they extended the attack to 9-round PRINCE which required about 2^{57} chosen plaintexts, 2^{64} encryptions and $2^{57.3}$ 64-bit memories. In FSE 2014, Canteaut *et al* [4] conducted a multiple differential cryptanalysis on a reduced round version of PRINCE. In 2015, Zhao *et al* [20] proposed a truncated differential cryptanalysis of PRINCE. Morawiecki [12] proposed different practical attacks on the round-reduced PRINCE. In FSE 2015, Derbez *et al* [6] proposed a new meet-in-the-middle attack and structural analysis of round-reduced PRINCE.

Recently, the Ruhr university is holding the PRINCE Challenge [14]. The task of this competition is to find practical attacks on round-reduced version of the block cipher PRINCE. The challenges come in two flavors, one in the known plaintext setting and the other in the chosen plaintext setting. For each setting, there are five challenges which differ by the number of rounds to be attacked. As shown in the pub-

lic official site [14], given 2^{20} chosen plaintexts/ciphertexts, Pawel *et al* could launch an attack using only 2^7 chosen plaintexts with a time complexity of 2^{11} for the 4 rounds of PRINCE. Patrick mentioned two more attacks. As for 6 rounds, he broke the cipher using 2^{16} chosen plaintexts with $2^{33.7}$ complexity. As for 8 rounds, he broke PRINCE using 2^{16} chosen plaintexts with about 2^{50} complexity.

All cryptanalyses aforementioned are at the theoretical level. Since the traditional cryptanalysis will be getting exponentially complicated with the increase of the number of rounds, all the attacks can only work on the reduced round version of PRINCE, not on the full round version.

After the advent of PRINCE, only a few of fault analyses have already been proposed, which adopted the similar technique from DFA on AES [13], [18]. In 2013, Song *et al* [15] proposed a DFA which was applied to PRINCE under a random nibble-level fault model. It was based on their observations on the linear layer. They could extract the 128-bit key using less than 7 fault injections on average. Jovanovic *et al* [8] proposed a multi-stage nibble-based fault attack on LED and PRINCE. The attack was composed of two stages. In the first stage, the adversary could recover the round key $k'_0 + k_1$ by analyzing the faults injected into the 9-th round. In the second stage, they used $k'_0 + k_1$ to peel off the last round and recover k_1 by analyzing the faults injected into the 8-th round. Their results showed that, for the same amount of fault injections, the reduced key search space in stage 1 and stage 2 are the same. Three fault injections (two in the 9-th round, one in the 8-th round) can reduce the key search space of the 128-bit key to $2^{41.85}$ on average.

1.3 Contribution and organization

In this paper, we evaluate the security of PRINCE against fault attacks with AFA techniques. Extensive experiments are conducted on PRINCE under simple nibble-based fault model, and more advanced fault models such as byte-based, half word-based and word-

This paper evaluates the security of PRINCE against fault attacks. Different from previous DFAs, the authors combine fault analysis with algebraic cryptanalysis, and conduct AFA on PRINCE. Extensive experiments are conducted on PRINCE.

based fault models.

Under nibble-based fault model, AFA can succeed with three or four fault injections within 300 seconds with a very high probability. Under other advanced fault models with more faulty bits, the fault propagation paths become overlapped in the last round and previous DFA attacks are difficult to work. Our results show that AFA can still succeed to recover the 128-bit master key of PRINCE.

To evaluate the security of PRINCE against fault attacks, we utilize AFA to calculate the reduced entropy of the secret key when given certain amount of fault injections. The results can be used to interpret previous work and compare the efficiency. Under nibble-based fault model, the 128-bit key of PRINCE can be reduced to $2^{9,69}$ and $2^{36,10}$ with 3 and 2 fault injections, respectively. These results show that, compared with DFA, AFA is much more generic, automatic, uncomplicated and trustful.

The rest of the paper is organized as follows. Section II describes the design of PRINCE. Section III presents the framework of AFA on block ciphers. Section IV presents the fault models. Section V fully discusses the

details of building the equations. Sections VI shows the experimental results. Section VII concludes the paper.

II. THE BLOCK CIPHER: PRINCE

PRINCE [7] is a 64-bit block cipher with a 128-bit key. In this paper, the master key is denoted as K . The plaintext and ciphertext of 64 bits each are denoted as P and C respectively. The high level structure of PRINCE is shown in Figure 1. PRINCE has two major components: the key expansion part and the encryption part. It is based on the so-called FX construction: the first two subkeys k_0 and k'_0 are used as whitening keys. The key k_1 is the 64-bit key for an inner block cipher, which is of 10 rounds and referred to as PRINCE_{core} .

2.1 Key expansion

K , the key of PRINCE is split into two parts of 64 bits each, denoted as k_0 and k_1 respectively.

$$K = k_0 || k_1 \quad (1)$$

Then K is extended to 192 bits by a linear mapping of form

$$(k_0 || k_1) \rightarrow (k_0 || k'_0 || k_1) \quad (2)$$

where k'_0 is defined as

$$k'_0 = (k_0 \ggg 1) \oplus (k_0 \ggg 63) \quad (3)$$

2.2 Encryption

The whole encryption process of PRINCE_{core} is depicted in Figure 2. Each round of PRINCE_{core} consists of four major components: a key addition (AK), an S-Box layer (S or S^{-1}), a



Fig.1 The high level structure of PRINCE

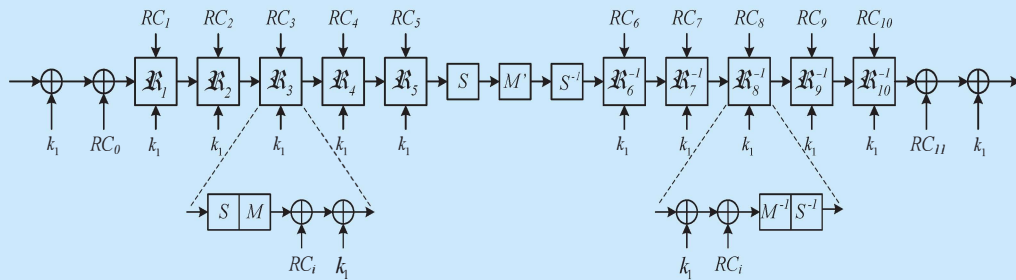


Fig.2 PRINCE_{core}

linear layer (M or M^1) and the addition of a round constant (RC).

In this paper, we clarify that a *word* is of 32 bits, a *half word* of 16 bits, a *byte* of 8 bits and a *nibble* of 4 bits. The intermediate computation result, called *state* is usually represented by a 64-bit vector or a 16-nibble vector. We denote the full state as X , or as $X_0, X_1, \dots, X_{16}$ if the nibble representation is preferred.

1) k_i -add: The 64-bit state is xored with the 64-bit subkey.

2) S-Layer S or S^1 : The cipher uses a 4-bit S-Box which is given as in Figure 3. We denote the S-Box and its inverse by S and S^1 respectively.

3) The Matrices: M or M^1 -Layer

The 64-bit state is multiplied with a 64×64 matrix, M or M^1 . Both are associated with a matrix M' used in the middle round. M' is built from four 4×4 matrices M_0, M_1, M_2, M_3 . The details of how to build M or M^1 can be referred to [7].

4) RC_i -add: In the i -th round, a 64-bit round constant is xored with the state.

III. THE OVERVIEW OF ALGEBRAIC FAULT ANALYSIS

In order to overcome the disadvantage of DFA and find a general way of key recovery, we conduct a comprehensive *algebraic fault analysis* (AFA) on PRINCE.

Figure 4 gives an illustration of the overall

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$S[x]$	B	F	3	2	A	C	9	1	6	7	8	0	E	5	D	4

Fig.3 The S-Box of PRINCE

framework of AFA. As shown in Figure 4, AFA usually contains three stages: *Fault injection*, *Equation representation* and *Equation solving*. In Figure 4, T_m and T_m^* are the intermediate states of normal and faulty encryption respectively. K_r is the round key in the next round of the one where faults are injected. $g(\cdot)$ is the encryption function after injecting the fault. $h(\cdot)$ is the key expansion function from the master key to the round key. $E(\cdot)$ stands for the function of the entire encryption.

3.1 Fault injection

A fault attack is to maliciously induce errors to devices in order to alter the execution of cryptographic algorithms inside. There are many practical methods of injecting faults, such as optical radiation injection, voltage supply or clock glitch, critical temperature injection and EM injection.

In this paper, we use some customized programs to simulate the fault injections at a software level. We will focus on the capability of algebraic analysis on PRINCE. The physical experiment in practice could be supported with some advanced platforms, such as SASE-

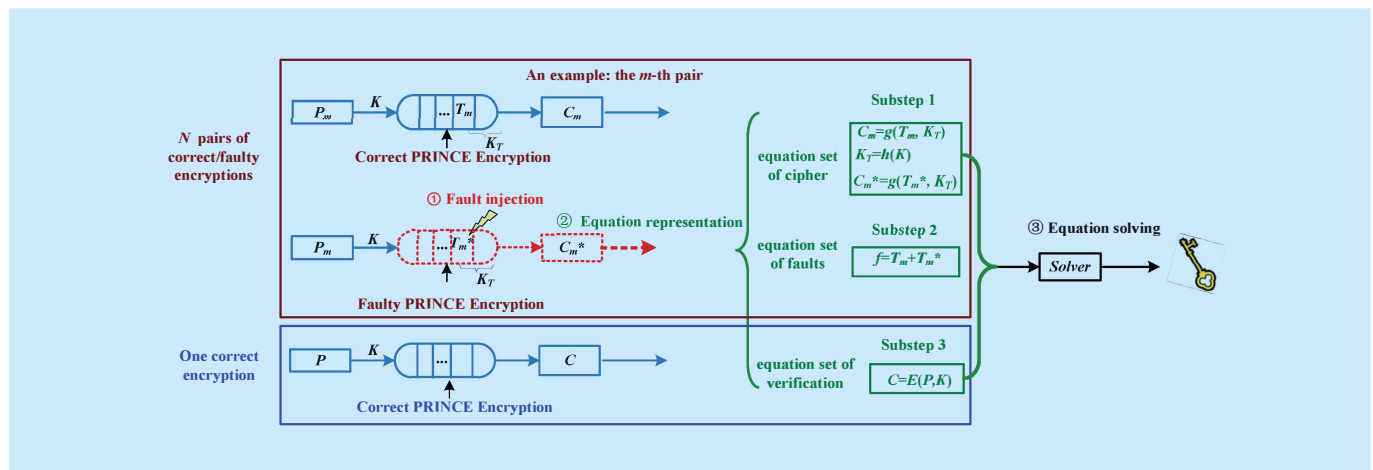


Fig.4 Framework of AFA on PRINCE

BO-W with clock glitching or Inspector with laser injections.

3.2 Equation representation

We use the plaintext P , the ciphertext C , some intermediate states (*i.e.* A , B , X , D , E introduced in later sections), and the key variables K to build the equations. The most essential and difficult part is how to represent the S-Box and the linear layer into algebraic equations. Since AFA mainly uses correct and faulty ciphertexts to recover the key, it is better to use ciphertexts to build equations reversely.

After the simulated fault injections, the faults are also presented with algebraic equations. The representations of faults will vary due to different scenarios, such as what type of fault model is using, or whether the locations of faults are known or not.

3.3 Equation solving

In this phase, the master key can be recovered by solving the combined equation set of the cipher and the fault differences. Many automatic tools, such as mutantXL algorithm and Gröbner basis-based, can be leveraged. We choose CryptoMiniSAT v2.9.4 [16], which won the Gold prizes in the SAT Race competition in 2010, as the solver in our analysis. In our experiment, the solver runs on a quad-core laptop with Intel Core I7-2640M, 2.80 GHZ, 4G memory and Windows 7 64-bit OS.

IV. AFA ON PRINCE: FAULT MODELS

We introduce the notations and the fault models before we move to the specific algebraic fault attack on PRINCE. When in the reverse direction, S^{-1} is used in the S-Layer and M^{-1} is used in the linear layer for late rounds. The order of those operations for PRINCE in a late round is: k_r -add, RC_r -add, M^{-1} and S^{-1} as shown in Figure 2.

4.1 Notations

We use A to denote the state for the output from the $(r-1)$ -th round. A is also a state for the key addition in the r -th round. We use B , X , D ,

E to denote the state after the key addition, the round constant addition, the linear layer M^{-1} and the S-Box respectively. We define the notation $A^{(r)}$, $B^{(r)}$, $X^{(r)}$, $D^{(r)}$, $E^{(r)}$ if a specific round r is referred.

For a specific state X before the linear layer M^{-1} , it can be represented as $x_0||x_1||\cdots||x_{63}$ where x_i is a bit. It can also be represented as $X_0||X_1||\cdots||X_{15}$ where X_i is a nibble. $X_i = x_{4+i}||x_{4+i+1}||x_{4+i+2}||x_{4+i+3}$. Note that X can also be represented as $X_0||X_1||\cdots||X_m$, where each X_i could be a byte, a half word or a word. Similarly we can have B_i , D_i , E_i and b_i , d_i , e_i respectively. Note that E is the round output in the r -th round. We can have $E^{(r)} = A^{(r+1)}$.

The linear layer M^{-1} is associated with a permutation SR , *i.e.* $M^{-1} = M \circ SR^{-1}$. SR^{-1} is the inverse of SR which behaves like the AES ShiftRows and permutes the 16 nibbles of the state as $(X_0, X_1, \dots, X_{15}) \rightarrow (X_0, X_5, \dots, X_{11})$, where the subscripts are changed according to a table in [7]. It is the linear layer M^{-1} that propagates the faults in different columns to the final ciphertexts. We first clarify some notations on the fault model.

- f_p : the *fault position*, refers to the round where faults are injected. As for PRINCE, $5 \leq f_p \leq 10$.
- f_w : the *fault width*, refers to the number of faulty bits. The common value of f_w might be 1, 4, 8, 16, 32, which denotes the single bit, nibble-based, byte-based, half word-based or word-based fault respectively.
- f_s : the *target state*, refers to 1-bit output of the targeted operation where faults are injected.
- f_c : the *target cell*, refers to f_w -bit cell in f_s where faults are injected.
- f_d : the *fault difference*, refers to difference of the correct and fault cell.
- f_l : the *fault location*, refers to the index of f_c in f_s . **Location** is referred to as the place where the fault injection is situated.

4.2 Simple fault model

In this case, we assume that an adversary A can inject a single nibble fault whose location

cannot be predicted in advance. In a simple model, the faults are injected to a specific round r . The effect of the introduced fault is to add an arbitrary nonzero nibble disturbance to the state. However, the location index of the fault nibbles is unknown.

For the simplicity, we assume faults are induced to X_i in the r -th round. $5 \leq r \leq 10$. Note that round key addition and the S-Box permutation are two operations that will only change the values of the states. They will not expand the faults to some other adjacent nibbles in the state. Figure 5(a) demonstrates the fault propagation procedure of one single nibble-based fault injected into the k_r -add or RC_r -add output in the 9-th round. In Figure 5(a), the linear layer M^1 is represented with SR^1 and M' because . A marked box is a place which will be affected by the injected fault along the propagation path. A box marked in grey means the value of fault difference remains unchanged. A box marked in other colors means the fault differences are altered.

As demonstrated in Figure 5(a), the difference of the injected fault remains unchanged until it goes into the M^1 layer in the 9-th round. The M' -mapping spreads the difference to a group of four nibbles, and then the four nibble differences are changed by the S^1 . In order to make a distinction among the four nibble differences, we color them differently. The SR^1 in the 10-th round will route the four nibble differences into four different words, making each word have one and only one nonzero nibble of difference. After that, the subsequent M' -layer propagates differences within words, resulting full difference in the final ciphertext.

4.3 Advanced fault model

In this case, we assume that an adversary A is able to inject one fault under more advanced models, such as byte-based, half word-based, and word-based fault models.

In fault attacks, when migrating from theoretical analysis down to the physical analysis, the width of the induced fault confines the capability of real injections. Faults with greater

width are usually much easier to generate with low cost experiment setup. However, under these advanced fault models, the propagation paths for different faults are also overlapped due to the design of PRINCE.

Next, we take byte-based fault model as an example to demonstrate its fault propagation. Figure 5(b) demonstrates the fault propagation procedure of one single byte-based fault injected into the k_r -add or RC_r -add output in the 9-th round. A marked box is a place which will be affected by the injected fault along the propagation path. A box marked in grey means the value of fault difference remains unchanged. Note that it now has two nibbles under byte-based fault model. A box marked in other colors means the fault differences are altered. Note that in the M' layer in the 9-th round, there are many faults that are overlapped. That's why these boxes are marked as blurred.

We can see that since the M' -mapping in the 10-th round, fault differences are overlapped. It's extremely difficult for traditional DFAs to derive the input differences of S^1 in the 10-th round. The technique in [8],[15] cannot be directly applied. A new manual analysis has to be customized for this byte-model fault injection, which is very difficult according to our experience. Note that, the analysis over those overlapped paths would be much more complicated if half word or full word based fault models are referred to.

V. AFA ON PRINCE: EQUATION CONSTRUCTIONS

The goal of this phase is to transform PRINCE into a large system of low degree boolean equations. Solving the system is equivalent to recovering the key bits.

5.1 Overview of equation construction algorithm

If a fault is injected in the deep rounds, the fault difference will extensively prorogate to ciphertexts, which will cause the manual analysis far beyond the adversary's capability. As

a result, previous fault injects are normally in the last few rounds. Since fault analysis starts from ciphertexts, it will be much faster to set up equations for the decryption of PRINCE rather than for the encryption. The detailed procedure is depicted in Algorithm 1.

First, the adversary A builds the equation set of the key expansion of PRINCE. Then he generates N number of random plaintexts, $\mathbb{P} = \{P_1, P_2, \dots, P_N\}$. For each plaintext P_i ($1 \leq j \leq N$), he should:

- Execute the PRINCE encryption on P_i and collect the correct ciphertext C_i (Line 9)
- Use C_i to build the equation set for the correct encryption from the last round to the f_p -th round (Line 10-13, decryption order)
- Inject faults into X_{f_p} during the encryption of PRINCE on P_i and collect the erroneous ciphertext C_i^* (Line 14)
- Use C_i^* to build the equation set for the faulty encryption from the last round to the

f_p -th round (Line 15-18, decryption order)

- Build the equation set for the faults $f_d = X_{f_p} + X_{f_p}^*$ (Line 19)

Additionally, he has to generate one random plaintext P_v for verification, execute the correct encryption, and collect the ciphertext C_v . The adversary has to use both P_v and C_v to build the equation set of one full round correct encryption of PRINCE (Line 21-26, decryption order).

5.2 Building the Equations for PRINCE

1) *Representing k_r -add*: A_i and B_i ($1 \leq i \leq 16$) denote one nibble of the input and output state in round key addition respectively. Let k_r -add denote one nibble in k_1 . Then AK is represented as

$$B_i = A_i + K_i \quad (4)$$

2) *Representing RC_r -add*: B_i and X_i ($1 \leq i \leq 16$) denote one nibble of the input and output state

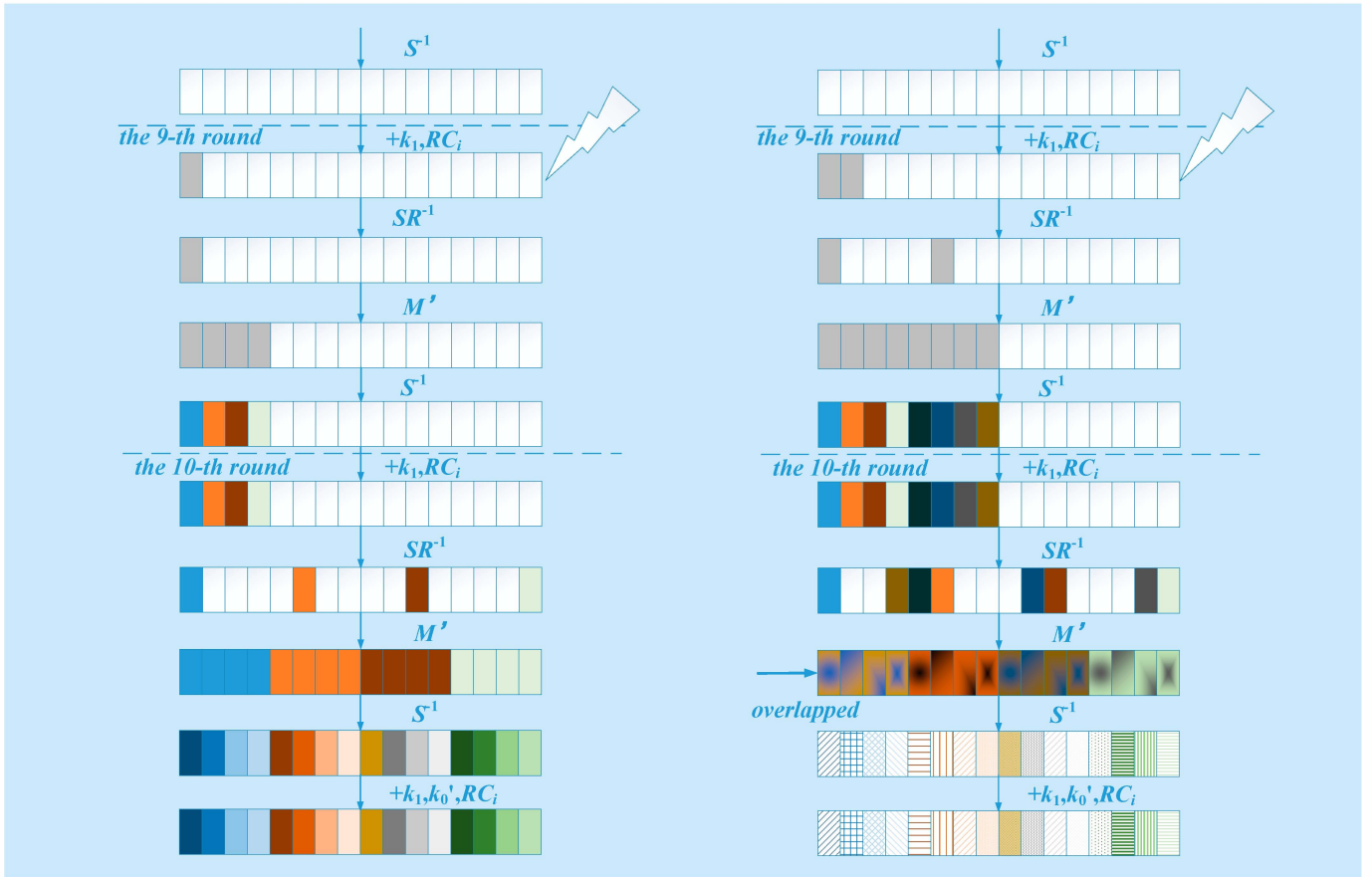


Fig.5 One fault injection to the ninth round
(a) nibble-based fault model

(b) byte-based fault model

of round constant addition in the r -th round respectively. Let $RC_r(i)$ denote one nibble in RC_r . Then RC_r -add can be represented as

$$X_i = B_i + RC_r(i) \quad (5)$$

3) *Representing S^{-1} and S* : In this paper, we adopt the techniques in [9] to derive every S-Box output bit with high-degree equations from the four S-Box input bits. Suppose (x_0, x_1, x_2, x_3) and (y_0, y_1, y_2, y_3) are the input and output of the 4-bit S-Box. Then, the S-Box lookup can be represented as

$$\begin{aligned} y_0 &= 1 + x_0 + x_2 + x_0x_1 + x_1x_2 + x_0x_1x_3 + \\ &\quad x_0x_2x_3 + x_1x_2x_3 \\ y_1 &= x_0 + x_3 + x_0x_2 + x_0x_3 + x_2x_3 + \\ &\quad x_0x_1x_2 + x_0x_2x_3 \\ y_2 &= 1 + x_0x_2 + x_1x_2 + x_1x_3 + x_0x_1x_2 + \\ &\quad x_1x_2x_3 \\ y_3 &= 1 + x_0 + x_1 + x_0x_1 + x_0x_3 + x_1x_2 + \\ &\quad x_2x_3 + x_1x_2x_3 \end{aligned} \quad (6)$$

The inverse of S, S^{-1} , can be represented as

$$\begin{aligned} y_0 &= 1 + x_2 + x_3 + x_0x_1 + x_1x_2 + x_1x_3 + x_2x_3 + \\ &\quad x_0x_1x_2 + x_0x_1x_3 + x_1x_2x_3 \\ y_1 &= x_1 + x_3 + x_0x_2 + x_1x_2 + x_1x_3 + x_2x_3 + \\ &\quad x_0x_2x_3 + x_1x_2x_3 \\ y_2 &= 1 + x_0x_1 + x_0x_2 + x_1x_2 + x_1x_3 + \\ &\quad x_1x_2x_3 \\ y_3 &= 1 + x_0 + x_0x_1 + x_1x_2 + x_2x_3 + \\ &\quad x_0x_1x_3 + x_0x_2x_3 \end{aligned} \quad (7)$$

4) *Representing M^{-1} and M* : PRINCE adopt two Matrix layers and the 64-bit state is multiplied with a 64×64 matrix, M or M' . There are different requirements for M and M' . The M' -layer is only used in the middle round, thus M' has to be an involution to ensure the α -reflection property. This requirement does not apply for the M -layer used in the round functions. Meanwhile, the designers of PRINCE want to ensure full diffusion after two rounds. Thus, M is designed by combining the M' -mapping with an application of matrix SR which behaves like the AES ShiftRows and permutes the 16 nibbles.

Thus $M = SR \circ M, M^{-1} = M \circ SR^{-1}$.

Suppose X_i and y_i ($0 \leq i \leq 63$) denote one bit input and output of SR . Let vector R and R^{-1} denote the shifted nibble index for SR and SR^{-1} respectively. $R = \{0, 5, 10, 15, 4, 9, 14, 3, 8, 13, 2, 7, 12, 1, 6, 11\}$, $R^{-1} = \{0, 13, 10, 7, 4, 1, 14, 11, 8$

Algorithm 1 The AFA procedure on PRINCE

```

1:  $rmax=32$ ;
2:  $N \leftarrow$ , input the number of faults;
3:  $f_p \leftarrow$ , input fault position;
4:  $f_w \leftarrow$ , input fault width;
5:  $b_f \leftarrow$ , input flags of  $f_w$  is known or not;
6: Generate_KS_PRINCE()
7: Random_Plaintext_Generating( $\mathbb{P}$ )
8: for  $i=0$ ;  $i < N$ ;  $i++$  do
9:    $C_i = E(P_r K)$ 
10:  for  $j=rmax$ ;  $j > f_p-1$ ;  $j--$  do
11:    Generate_Encrypt_Round_ES()
12:  end for
13:  Generate_Input_ES( $C_i$ )
14:  Fault_Inducing( $E(P_r K)$ )
15:  for  $j=rmax$ ;  $j > f_p-1$ ;  $j--$  do
16:    Generate_Encrypt_Round_ES()
17:  end for
18:  Generate_Input_ES( $C_i^*$ )
19:  Generate_Fault_ES( $f_d = X_{f_p} + X_{f_p}^*$ )
20: end for
21: Verify_Plaintext_Generating( $P_v$ )
22: for  $j=0$ ;  $j < rmax$ ;  $j++$  do
23:  Generate_Encrypt_Round_ES()
24: end for
25: Generate_Input_ES( $P_v$ )
26: Generate_Input_ES( $C_v$ )

```

, 5, 2, 15, 12, 9, 6, 3}. Then SR can be represented as

$$y_i = x_{(4 \times R[i/4]) + i \% 4} \quad (8)$$

SR^{-1} can be represented as

$$x_i = y_{(4 \times R^{-1}[i/4]) + i \% 4} \quad (9)$$

where % denotes the modular function.

Suppose X_i and y_i ($0 \leq i \leq 63$) denote one bit of the input and output of M' . y_i can be represented by simply xoring some variables of X_i . Let $\phi(i)$ denote the index set of X_i in representing y_i . Table 1 shows $\phi(i)$ used in M' . For instance, $y_0 = x_4 + x_8 + x_{12}$, and $y_{41} = x_{33} + x_{37} + x_{45}$.

From Table 1, we can see that the branch number of M' is 4. If a single nibble-based fault is injected at the input of M' , then at least three at most four nibbles become faulty. Specifically speaking, if the Hamming weight of the input fault difference is one, then three nibbles become faulty. if the Hamming weight of the input fault difference is larger than one,

then four nibbles become faulty.

5.3 Building the equations for faults

Let X denote the 64-bit correct state of PRINCE. $X = x_1 || x_2 || \dots || x_{64}$. X is the state before the linear layer M^1 . Let Y denote the faulty value of X after fault injections. $Y = y_1 || y_2 || \dots || y_{64}$.

Let f be denoted as the fault itself, w as the width of f , and t as the location of f . As for the specific r -th round, there are M possible locations for the injected faults where $m=64/w$. Let Z denote the fault difference of X and Y , which is

$$Z = z_1 || z_2 || \dots || z_b, z_i = x_i + y_i, 1 \leq i \leq 64 \quad (10)$$

Then, Z can be divided into M parts denoted as $Z_1 || Z_2 || \dots || Z_m$.

$$Z_i = z_{w \times (i-1)+1} || z_{w \times (i-1)+2} || \dots || z_{w \times i}, 1 \leq i \leq m \quad (11)$$

According to whether or not A knows the exact location t ($1 \leq t \leq m$) of the specific fault f , the algebraic equation representation of Z may have different formats.

1) *The fault location t is known:* Suppose t is known. Then Z can be denoted as

$$Z_i = 0, 1 \leq i \leq m, i \neq t \quad (12)$$

Obviously, Z_t has a nonzero value of w -bits. We introduce a single bit variable u_t to represent that Z_t is faulty.

$$u_t = (1 \oplus z_{w \times (t-1)+1}) (1 \oplus z_{w \times (t-1)+2}) \cdots (1 \oplus z_{w \times t}) = 0 \quad (13)$$

2) *The fault location t is unknown:* In practice, the value of t may not be pre-determined. We introduce a variable u_i of m bits to represent whether Z_i is faulty or not.

$$u_i = (1 \oplus z_{w \times (i-1)+1}) (1 \oplus z_{w \times (i-1)+2}) \cdots (1 \oplus z_{w \times i}), 1 \leq i \leq m \quad (14)$$

If $u_i = 0$, Z_i will be the variable that is associated with the injected w -bit fault. Assume there is one and only one fault is injected, there should be only one zero among $u_1 || u_2 || \dots || u_m$. This constraint can be represented as:

$$(1 - u_1) \vee (1 - u_2) \vee \cdots \vee (1 - u_m) = 1, \quad (15)$$

$$u_i \vee u_j = 1, 1 \leq i < j \leq m$$

The fault difference of PRINCE can be fully represented with Equations 10-11 or 12-15. Note that these equations can also be represented when a different w is given.

VI. EXPERIMENTAL RESULTS

There are two modes of conducting the simulations using CryptminiSAT. The first one, denoted as Mode A, is with a verification stage. It is built for the last few rounds of PRINCE to enforce the number of solutions to be one all the time. The purpose is to get the statistics of different solving time with different number of fault injections. The second one, denoted as Mode B, is without the verification stage. The solver is running at a multiple solution mode. Given the number of fault injections, it can help get the statistics of the number of the solutions and the reduced key search space.

Let N_1 and N_2 denote the number of fault injections to the linear layer M^1 in the ninth and eighth round, respectively. Then, the total number of fault injections can be denoted as $N = N_1 + N_2$. Let $\phi(K)$ denote the entropy of secret key K . $\phi(K)$ also represents the remaining unexploited key bits under the key search space. $\phi(K) = 0$ denotes that K is fully recovered.

6.1 AFA on PRINCE under simple fault model

In this section, we conducted AFA under the

Table 1 $\phi(i)$ used in M'

i	$\phi(i)$	i	$\phi(i)$	i	$\phi(i)$	i	$\phi(i)$
0	4,8,12	16	16,20,24	32	32,36,40	48	52,56,60
1	1,9,13	17	21,25,29	33	37,41,45	49	49,57,61
2	2,6,14	18	18,26,30	34	34,42,46	50	50,54,62
3	3,7,11	19	19,23,31	35	35,39,47	51	51,55,59
4	0,4,8	20	16,20,28	36	32,36,44	52	48,52,56
5	5,9,13	21	17,21,25	37	33,37,41	53	53,57,61
6	2,10,14	22	22,26,30	38	38,42,46	54	50,58,62
7	3,7,15	23	19,27,31	39	35,43,47	55	51,55,63
8	0,4,12	24	16,24,28	40	32,40,44	56	48,52,60
9	1,5,9	25	17,21,29	41	33,37,45	57	49,53,57
10	6,10,14	26	18,22,26	42	34,38,42	58	54,58,62
11	3,11,15	27	23,27,31	43	39,43,47	59	51,59,63
12	0,8,12	28	20,24,28	44	36,40,44	60	48,56,60
13	1,5,13	29	17,25,29	45	33,41,45	61	49,53,61
14	2,6,10	30	18,22,30	46	34,38,46	62	50,54,58
15	7,11,15	31	19,23,27	47	35,39,43	63	55,59,63

same nibble-based fault model mentioned in [8],[15]. The work in [8],[15] divides the key recovery of fault attacks on PRINCE into two stages. In the first stage (denoted as Stage 1), the faults are injected into the 9-th round to recover $k_1 + k'_0$ and the remaining entropy of $k_1 + k'_0$ is calculated. In the second stage (denoted as Stage 2), assuming $k_1 + k'_0$ has been fully recovered, the faults are injected into the 8-th round to obtain the remaining entropy of k_1 . From the remaining entropy of $k_1 + k'_0$ and k_1 in the two phases, when $N_1=2$, there is no evidence that they have conducted the real key recovery attack by combining the two stages. Different from [8],[15], our AFA is an all-in-one attack in the analysis procedure where the second stage is directly associated with the result from the first stage, not just based on an assumption of the first stage. Our technique can fulfill the real key recovery attacks under the same setting as [8],[15].

1) AFA under $N_1=2$ and $N_2=2$: We first run 100 instances under Mode A where $N_1=2$ and $N_2=2$. An *instance* means that we feed all the equations that generated according to Section V into the CryptminiSAT and run the solver for one time. The time-out of the solver is denoted as t_{out} and the success rate is denoted as S_{rate} . We calculate different solving times as shown in Figure 6(a).

In Figure 6(a), the y-axis is the percentage of successful instances. The X-axis is the

solving time in seconds. We can see that the statistics seems to follow an exponential distribution. The full master key can be solved within 60 seconds at a success rate of 86%. In fact, if we set the time-out of the solver as 300 seconds, the success rate is 100% in our experiments.

2) AFA under $N_1=2$ and $N_2=1$: Then, we also run 100 instances under Mode A where $N_1=2$ and $N_2=1$. The time statistics is shown in Figure 6(b).

Figure 6(b) demonstrates that the full master key of PRINCE can be solved within 300 seconds at a success rate of 80%. It seems that three fault injections can reduce the master key into a very limited search space. For detailed estimation on the reduced key search space, we will address this later in this section.

In this section, we can verify that AFA is able to combine the analysis of the two stages in [8],[15] into one and to recover the master key of PRINCE with at least 3 faults under nibble-based fault model with a high probability ($S_{rate} \geq 80\%$ when $t_{out} = 300$).

6.2 AFA on PRINCE under advanced fault models

In physical fault attacks, fault models with a wider fault with usually are much easier to general with low cost experiment setup. However, as mentioned in Section IV, under these advanced fault models, the propagation paths

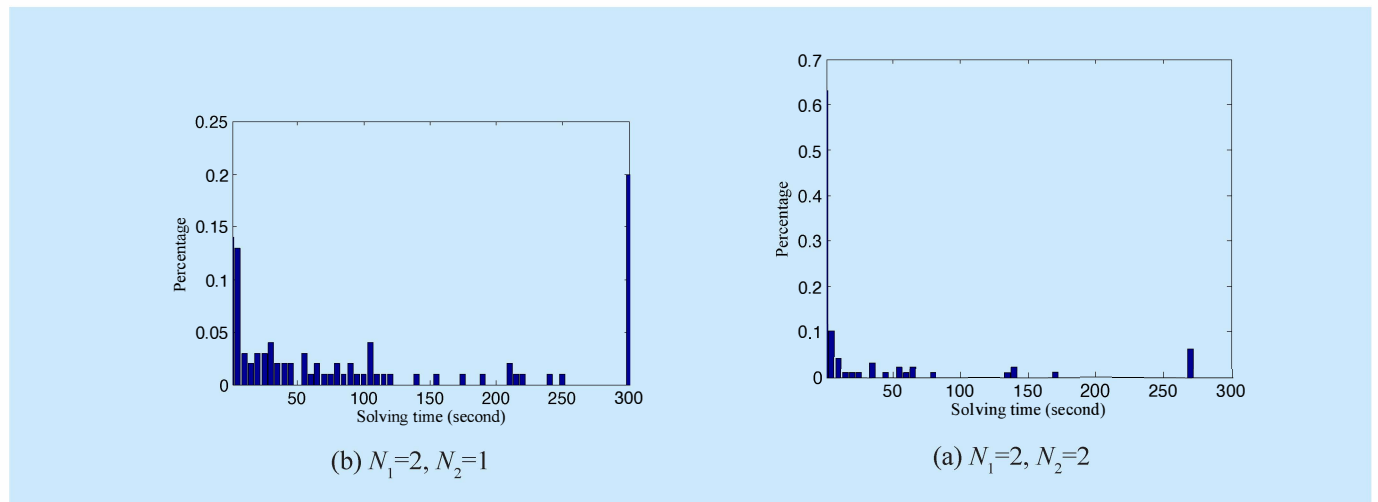
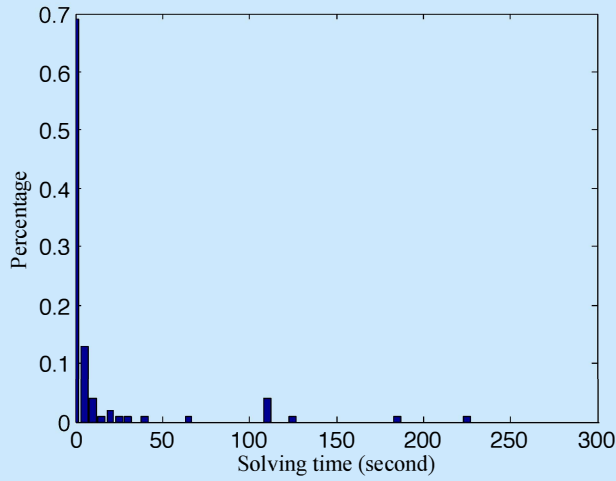
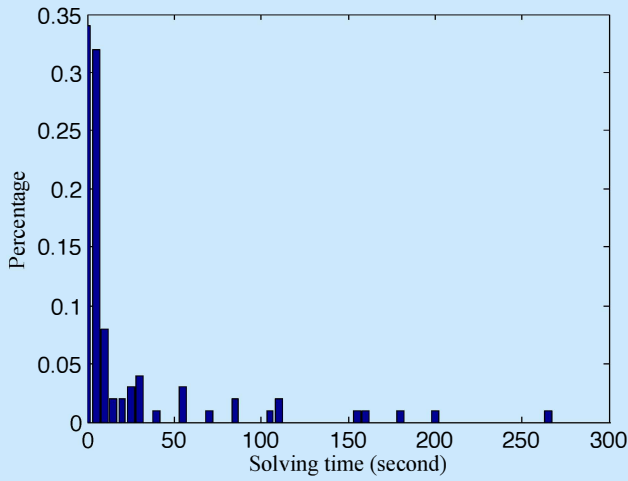


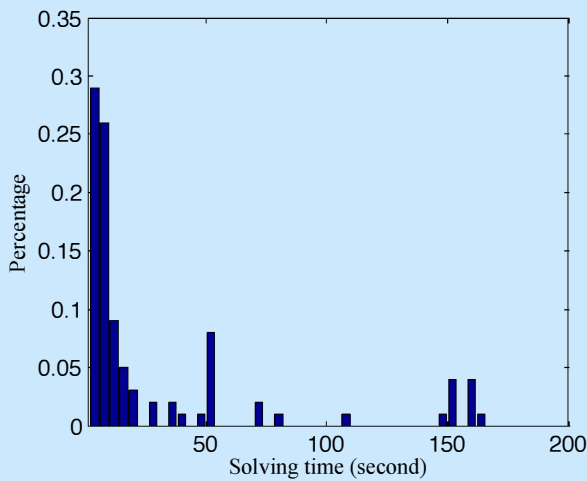
Fig.6 Equation solving time under Simple Fault model (Mode A)



(a) byte-based fault model



(b) half word-based model



(c) word-based fault model

Fig.7 Equation solving time in AFA on PRINCE under Advanced Fault models (Mode A, $N_1=2, N_2=2$)

are overlapped. It is difficult for traditional DFAs to work. However, AFA can well bridge this gap and achieve good performance.

With byte-based, half word-based, word-based fault models, we set $N_1=2, N_2=2$ under Mode A and calculate the different solving times. The statistics is shown in Figure 7. We can see that the correct key can be solved within 300 seconds at a success rate of 100%. In Figure 7, most of the instances are completed within 50 seconds. In another word, if we set t_{out} to 50 seconds, S_{rate} will still be close to a very high probability (around 90%).

6.3 Evaluating $\phi(k_1 + k'_0)$ and $\phi(k_1)$ with AFA

The work in [8],[15] applied DFA to estimate $\phi(k_1 + k'_0)$ in Stage 1 and $\phi(k_1)$ in Stage 2 separately. And their results are different, especially in Stage 2, as shown in Table II.

To interpret and analyze the results above, we apply AFA to evaluate the reduced key search space of PRINCE under given amount of faults in each phase.

1) *Evaluating $\phi(k_1 + k'_0)$ in Stage 1:* As in [8],[15], faults are injected into the RC_i -add of the 9-th round. We evaluate the reduced entropy of $k_1 + k'_0$ by analyzing the faults in the 10-th round with AFA. The algebraic equations of the correct and fault PRINCE encryption from the RC_i -add of the 10-th round to the ciphertext are built. Both the correct and faulty ciphertext are fed into the solver. Meanwhile, the algebraic equations of random 16-bit faults are also represented with the technique in Section V. Different from the previous subsection, we run AFA on PRINCE under Mode B for different value of N_1 .

In the attack, we have found that if N_1 is small, $\phi(k_1 + k'_0)$ is very large and it is difficult for the solver to finish the solving process within an affordable time-out. For example, when $N_1=1$, $\phi(k_1 + k'_0)$ might be around 34, the solver has to enumerate about 2^{34} solutions. To handle with this situation, we have written a program to feed random κ bits of $\phi(k_1 + k'_0)$ into the algebraic equations. And κ is chosen from a large value (for instance, 64)

to a small one. Let $\eta(\kappa)$ denote the number of solutions for given κ , and τ a threshold for the number of solutions. For example, if $\tau=2^{14}$, $\kappa=8$, $\eta(\kappa)>14$, then $\phi(k_1+k'_0)$ can be estimated as $64-8+\log_2(\eta(8))$. If $\kappa=0$ (it means no key bit is fed into the solver), $\eta(0)<\tau$, then $\phi(k_1+k'_0)=\log_2(\eta(0))$. The pseudo-code of the program is shown in Algorithm 2.

The results of 100 AFA instances are shown in Figure 8. $\phi(k_1+k'_0)$ can be reduced to 32.71, 11.22, 2.94, 0.98 for $N_1=1,2,3,4$ respectively.

From Figure 8, we can see that $\phi(k_1+k'_0)$ has a large range (from 17 to 50). It is caused by the small branch number of M' in PRINCE. According to Section V, if the Hamming weight of the input fault difference is 1, then after the M' layer in the 9-th round, at least 3 nibbles are faulty. After the M' layer in the 10-th round, about 9 to 12 nibbles in the ciphertexts are faulty. In this case, only 9 to 12 nibbles of $\phi(k_1+k'_0)$ can be utilized for the key recovery and $\phi(k_1+k'_0)$ is larger. If the Hamming weight of the input fault difference is larger than one, about 12 to 16 nibbles in the ciphertexts are faulty. Accordingly, 9 to 12 nibbles of $\phi(k_1+k'_0)$ can be used for the key recovery, resulting in a smaller value of $\phi(k_1+k'_0)$.

2) *Evaluating $\phi(k_i)$ in Stage 2*: Then, we apply the similar technique to estimate $\phi(k_i)$ in Stage 2. As the same as that in [8],[15], we assume that $k_1+k'_0$ has been fully recovered in Stage 1, $\phi(k_1+k'_0)=0$. In Stage 2, nibble-based faults are injected into the 8-th round of PRINCE. For $N_2=1,2,3,4$, we run 100 random AFA instances, $\phi(k_i)$ can be reduced to 21.25, 2.45, 0.28, 0, respectively. The statistics on $\phi(k_i)$ is shown in Figure 9.

3) *Discussions*: From the above two sections, we can see that one fault injection in the 8-th round can retrieve more key bits than in the 9-th round. This is because that, when one fault is injected into the 9-th round, only partial bits of $k_1+k'_0$ can be recovered by analyzing the 10-th round of PRINCE. k_1 cannot be directly obtained by analyzing the 9-th round. While this is not the same case in fault attacks in Stage 2. When $k_1+k'_0$ has been fully recovered

Table II $\phi(k_1+k'_0)$ and $\phi(k_i)$

work	$\phi(k_1+k'_0)$, Stage 1	$\phi(k_i)$, Stage 2
faultnumber	1, 2, 3, 4	1, 2, 3, 4
[22]($f_w=4$)	30.89,11.44,4.12,1.47	30.41,11.64,4.44,1.82
faultnumber	1, 2, 3, 4	1, 2, 2.56
[37]($f_w=4$)	34.76,14.18,4.86,1.78	25.32,3.01,1
faultnumber	1, 2, 3, 4	1, 2, 3, 4
AFA($f_w=4$)	32.71,11.22,2.94,0.98	21.25,2.45,0.28,0
faultnumber	1, 2, 3, 4	1, 2, 3, 4
AFA($f_w=8$)	31.44,8.08,0.89,0.16	19.21,1.21,0.16,0

Algorithm 2 Estimate $\phi(k_1+k'_0)$ by feeding random κ bits

```

1: for  $i=0; i<N; i++$ 
2:   Generate_AFA_ES();
3:   Generate_KNOWN_KEY_SET(64);
4:   for  $j=64; j\geq -1; j--$  do
5:     Generate_KNOWN_KEY_ES(j);
6:     Run_AFA_TYPEB();
7:     Calculate_AFA( $\eta(j)$ );
8:     if  $\eta(j)\geq\tau$ 
9:        $\phi(k_1+k'_0)=64-i+\log_2(\eta(j)/\tau)$ 
10:      break;
11:    end if
12:    Reduce_KNOWN_KEY_SET(1);
13:    if  $j=-1$  and  $\eta(j)<\tau$ 
14:       $\phi(k_1+k'_0)=\log_2(\eta(j))$ 
15:    end if
16:  end for
17: end for

```

and one fault is injected to the 9-th round of PRINCE, the 10-th round of PRINCE can be peeled off, partial bits of k_1 can be obtained by analyzing the fault propagation in the 9-th round. Since the same round key is used in both the 8-th and the 9-th round, the value of $\phi(k_1)$ can be further reduced by analyzing the faults in the 8-th round.

Compared with AFA on other AES-like block ciphers, e.g., LED (If single nibble-based fault or byte-based fault is injected into the 30-th round, the key entropy of LED is reduced to 12.9 and 9.9 respectively, which is much smaller than result of AFA on PRINCE.), the secret key of PRINCE is decreased much slower under the same fault

model. This is because that different from PRINCE using two independent keys ($k_1 + k'_0$, k_1) in Stage 1 and two identical keys in Stage 2 (k_1), only one 64-bit key is used in the post-whitening operation of LED and no round key is used in the last three rounds. Thus, the fault analysis result of the last round can be directed applied to analyze the penultimate round. In addition, the attacking result of the penultimate round can also be used to improve the fault analysis of the last round and further reduce the search space of the master key.

Meanwhile, we also evaluate $\phi(k_1 + k'_0)$ and $\phi(k_1)$ under byte-based fault model. The results (Table II) are even better than nibble-based fault model.

6.4 Evaluating $\phi(K)$ with AFA

In the previous subsection, we evaluate $\phi(k_1 + k'_0)$ and $\phi(k_1)$ separately in different stages. In fact, AFA can combine the two stages together and directly evaluate $\phi(K)$ under Mode B for given amount of fault injections.

Firs, we start a simple case where $N_1=2$ and $N_2=2$. We run 100 instances under Mode B. The result is shown in Figure 10. Under

nibble-based fault model, $\phi(K)$ can be reduced to less than 16 (4.16 on average). Under byte-based fault model, $\phi(K)$ can be reduced to less than 13 (2.74 on average), which is much less than that under nibble-based fault model. Under half word-based fault model and word-based fault model, $\phi(K)$ can be reduced to 13 (9.69 on average) and 18 (9.88 on average) respectively.

It should be noted that, under nibble-based fault model, we can see that $\phi(K)=4.16$ is much smaller than the case that setting $\phi(k_1 + k'_0)=11.22$ ($N_1=2$) in Stage 1 followed by setting $\phi(k_1)=2.45$ ($N_2=2$) in Stage 2. This is because that the solver is intelligent and automatic. It can recover k_1 by analyzing the faults in the 9-th round with limited candidates of $k_1 + k'_0$.

Then, we continue with a complicated case of AFA where $N_1=2$, $N_2=1$. Under nibble-based fault model, $\phi(K)$ can be reduced to 13 on average. This can be used to explain why the solver can retrieve K so fast under Mode A in Section VI when $N_1=2$ and $N_2=1$. Under nibble-based fault model, we also estimate $\phi(K)$ under $N_1=2$ and $N_2=1$, $\phi(K)$ can be

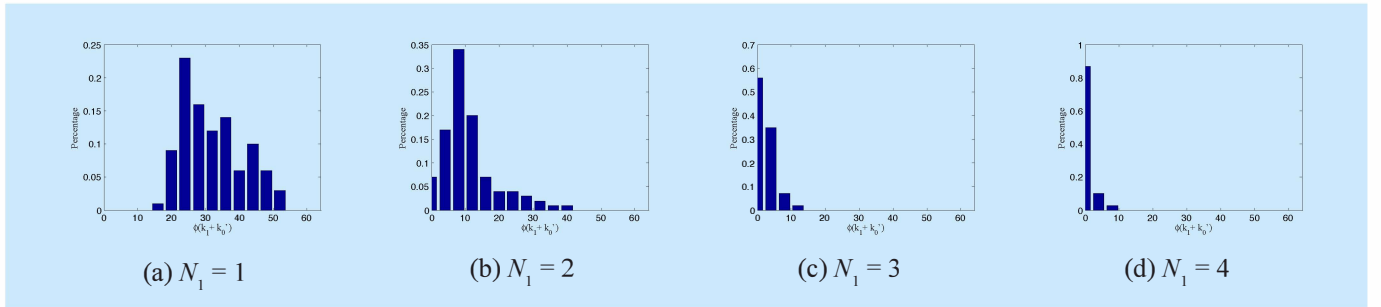


Fig.8 $\phi(k_1 + k'_0)$ under nibble-based fault model

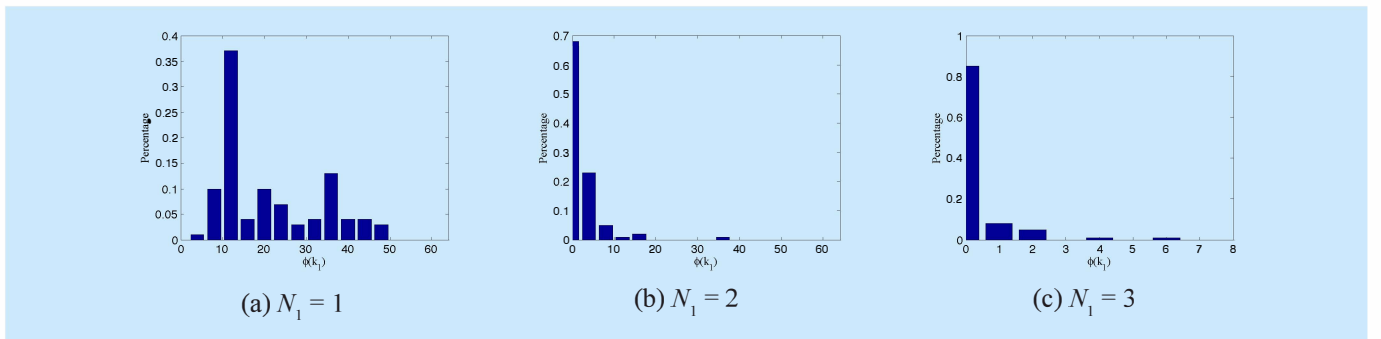


Fig.9 $\phi(k_1)$ under nibble-based fault model

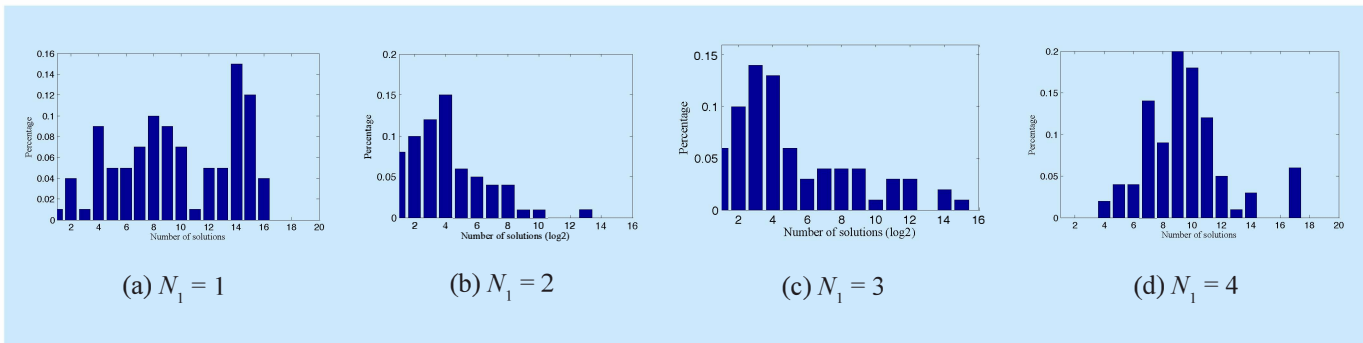


Fig.10 $\phi(K)$ when $N_1 = 2$, $N_2 = 2$

reduced to 36.10 on average.

VII. CONCLUSION

This paper evaluates the security of PRINCE against fault attacks. Different from previous DFAs, we combine fault analysis with algebraic cryptanalysis, and conduct AFA on PRINCE. Extensive experiments are conducted on PRINCE. The results show that with three or four fault injections, AFA can succeed within affordable time under nibble-based fault model. Meanwhile, AFA can also work under other fault models with large fault width, such as byte-based, half word-based, word-based fault models. To interpret previous DFAs on PRINCE, we apply AFA to evaluate the reduced key search space of PRINCE for given amount of fault injections. The results can be used to analyze and compare the efficiency of previous work. We illustrate that, for fault analysis on lightweight block ciphers, AFA is much more generic, automatic, uncomplicated and trustful than DFA. This work could be extended and provide some certain guidance for the construction of lightweight block ciphers with provable security against algebraic fault analysis.

ACKNOWLEDGMENT

The authors would like to thank the anonymous referees for helpful discussions and comments. This work was supported in part by the Major State Basic Research Development Program (973 Plan) of China under the

grant 2013CB338004, the National Natural Science Foundation of China under the grants 61173191, 61271124, 61272491, 61309021, 61472357, by the Zhejiang Provincial Natural Science Foundation of China under the grant LY13F010001, and by the Fundamental Research Funds for the Central Universities under the grant 2015QNA5005.

References

- [1] H. Bar-El, H. Choukri, D. Naccache, M. Tunstall, C. Whelan. The Sorcerer's Apprentice Guide to Fault Attacks. In IEEE 94, pp.370-382, 2006.
- [2] E. Biham, A. Shamir. Differential Fault Analysis of Secret KeyCryptosystem. In Proceedings of CRYPTO 1997, LNCS, vol. 1294, pp.513-525, 1997.
- [3] D. Boneh, R.A. DeMillo, R.J. Lipton. On the Importance of Checking Cryptographic Protocols for Faults. In Proceedings of EUROCRYPT1997, LNCS, vol. 1233, pp. 37-51, 1997.
- [4] A. Canteaut, T. Fuhr, H. Gilbert, M. Plasencia, J. Reinhard, Multiple. Differential Cryptanalysis of Round-Reduced PRINCE (FSE 2014, available at eprint 2014/089)
- [5] N. Courtois, D. Ware, K. Jackson. Fault-Algebraic Attacks on InnerRounds of DES. In Proceedings of eSmart 2010, pp. 22-24, 2010.
- [6] P. Derbez, L. Perrin,. Meet-in-the-Middle Attacks and Structural Analysis of Round-Reduced PRINCE (FSE 2015, available at eprint 2015/239)
- [7] B. Julia, C. Anne, et. al. In Proceedings of PRINCE - A Low-Latency Block Cipher for Pervasive Computing Applications. ASIACRYPT 2012, LNCS, vol. 7658, pp. 208-225, 2012.
- [8] P. Jovanovic, M. Kreuzer, I. Polian., Multi-Stage Fault Attacks on Block Ciphers (available at eprint 2013/778)
- [9] L.R. Knudsen, C.V. Miolane. Counting equations in algebraic attacks on block ciphers. International Journal of InformationSecurity, vol. 9, No. 2, pp. 127-135, 2010.

- [10] L. Li, K. Jia, X. Wang., Improved Meet-in-the-Middle Attacks on AES-192 and PRINCE (available at eprint 2013/573)
- [11] M. Mohamed, S. Bulygin and J. Buchmann. Improved Differential Fault Analysis of Trivium. In COSADE 2011, pp. 147-158, 2011.
- [12] P. Morawiecki, Practical Attacks on the Round-reduced PRINCE (available at eprint 2015/245)
- [13] Debdeep Mukhopadhyay. An Improved Fault Based Attack of the Advanced Encryption Standard. In AFRICACRYPT 2009 LNCS, vol. 5580, pp. 421-434, 2009.
- [14] http://www.emsec.rub.de/research/research_startseite/prince-challenge/
- [15] L. Song, L. Hu., Differential Fault Attack on the PRINCE Block Cipher (available at eprint 2013/043)
- [16] M. Soos, K. Nohl, and C. Castelluccia. Extending SAT Solvers to Cryptographic Problems. In Proceedings of SAT 2009, LNCS, vol. 5584, pp. 244-257, 2009.
- [17] J. Takahashi and T. Fukunaga. Improved Differential Fault Analysis on CLEFIA. In Proceedings of FDTC 2008, pp. 25-34, 2008.
- [18] M. Tunstall, D. Mukhopadhyay, S. Ali. Differential fault analysis of the advanced encryption standard using a single fault. In Proceedings of WISTP 2011, LNCS, vol. 6633, pp. 224-233, 2011.
- [19] F. Zhang, X.J. Zhao, S.Z. Guo, *et al.* Improved Algebraic Fault Analysis: A Case Study on Piccolo and Applications to Other Lightweight Block Ciphers. In Proceedings of COSADE 2013, LNCS, vol. 7864, pp. 62-79, 2013.
- [20] G. Zhao, B. Sun, C. Li, J. Su., Truncated differential cryptanalysis of PRINCE (Security and Communication Networks, 2015)

Biographies

ZHANG Fan, was born in 1978. He received his Ph.D. degree in the Department of Computer Science and Engineering from University of Connecticut in 2012. His research interests include side channel analysis and fault analysis in cryptography, computer architecture, security in wireless sensor network, and more. Currently he is working in the Department of Information Science & Electrical Engineering, Zhejiang University, China.

ZHAO Xinjie, was born in 1986. He received his Ph.D., M.S. and B.S. degrees in Department of Infor-

mation Engineering, Ordnance Engineering College in 2012, 2009 and 2006, respectively. He is currently an Engineer in Institute of North Electronic Equipment. His main research interest includes side channel analysis, fault analysis and combined analysis of block ciphers. He won the best paper award in Darmstadt - the 3rd International Workshop on Constructive Side-Channel Analysis and Secure Design (COSADE 2012).

GUO Shize, was born in 1964. He received his Ph.D. degree in Harbin Institute of Technology in 1989 and his M.S. and B.S. degrees from Ordnance Engineering College, China, in 1991 and 1988, respectively. He is currently a researcher in Institute of North Electronic Equipment and also a Professor in Beijing University of Post and Telecommunications. His main research interest includes information security and cryptography.

SHEN Jizhong, received the B.S. degree in Physics and the M.S. degree in Electrical Engineering from Hangzhou University, Hangzhou, China, in 1985 and 1992, respectively. In 2001, he received the Ph.D. degree in Electrical Engineering from Zhejiang University, Hangzhou, China. Currently, he is a full Professor in the Department of Information Science and Electronic Engineering, Zhejiang University, Hangzhou, China. Professor Shen is the author/co-author of more than 150 refereed technical papers on low-power Digital circuits design, brain computer interface (BCI) and so on. His current research interests include Digital integrated circuit & low power design, security in wireless sensor network, and Brain-Computer Interface and general information security. correspondence author: Telephone number: (+86)-571-87952886, E-mail: jzshen@zju.edu.cn .

HUANG Jing, was born in 1983. She received her M.S degree from Department of Computer Science, Tsinghua University University, China in 2013. She is currently an assistant researcher in the System Control Department of No.61541 Unit. Her research interests include side channel analysis and computer security.

HU Zijie, was born in 1993. He was admitted to the Department of Information Science & Electrical Engineering, Zhejiang University, China in 2011. His research interests include side channel analysis and general computer security. He will continue his graduate studies in Zhejiang University.